

ENTORNOS DE DESARROLLO

TRIMESTRE 2

DBML

DBML (Database Markup Language) es un lenguaje de marcado de código abierto, diseñado para ser simple y legible, cuyo objetivo es definir esquemas de bases de datos mediante código.

- Facilita el diseño visual (diagramas Entidad-Relación) antes de pasar a la programación real en SQL.
- Filosofía: "Documentación como código". Es fácil de compartir y versionar (en Git).

DBDIAGRAM.IO

Dbdiagram.io es una herramienta web (IDE especializado) que interpreta el código DBML y lo dibuja en tiempo real. Si escribes código a la izquierda el diagrama se genera a la derecha. Permite subir archivos .sql y convertirlos a diagramas. Genera el código SQL listo para usar en PostgreSQL, MySQL o SQL Server. Genera enlaces públicos para trabajar en equipo.

RUNSQL.COM

runsql.com es un entorno de ejecución y testeo web que permite validar bases de datos mediante SQL real sin necesidad de configurar servidores locales. Su interfaz se organiza en cuatro áreas clave: a la izquierda se define el esquema y se visualiza el contenido de las tablas, mientras que a la derecha se redactan las consultas y se comprueban los resultados obtenidos. Para que esta herramienta sea eficaz, el flujo de trabajo profesional exige introducir primero datos de prueba (datos dummy) en el apartado del esquema,

permitiendo así testear la lógica de las relaciones y la eficacia de las sentencias antes de implementarlas en un entorno de producción real.

QA, QC Y QE

El Quality Assurance (Garantía de Calidad) es un proceso proactivo y orientado al proceso. Su objetivo principal es prevenir defectos estableciendo estándares, metodologías y procedimientos sistemáticos (como auditorías y formación) antes de que el software se cree. Se centra en mejorar el proceso de desarrollo para asegurar que los resultados sean de alta calidad desde el inicio.

El Quality Control (Control de Calidad) es un proceso reactivo y orientado al producto. Su función es identificar y corregir defectos del producto final o en sus componentes ya desarrollados a través del testeo, inspección de código y seguimiento de bugs. Su objetivo es validar que el software cumple con los requisitos y especificaciones técnicas antes de su entrega.

El Quality Engineering (Ingeniería de Calidad) es una disciplina integral que combina QA y QC con metodologías modernas. Se enfoca en incorporar la calidad en todo el ciclo de vida del desarrollo de forma continua, apoyándose fuertemente en la automatización de pruebas y la integración continua. Su propósito es que la calidad sea una responsabilidad compartida y escalable desde la planificación hasta el mantenimiento.

En resumen: QA previene errores mejorando el proceso, QC detecta errores probando el producto, y QE automatiza e integra la calidad en todo el ciclo de desarrollo.

USER INTERFACE / USER EXPERIENCE

User Interface se centra en el cómo y en la apariencia. Es la parte visual y táctil con la que el usuario interactúa directamente. Incluye todos los elementos gráficos como botones, iconos, tipografías, colores y el diseño de la pantalla. Su objetivo es crear una interfaz que sea estéticamente atractiva y que guíe al usuario visualmente, asegurándose que el diseño sea coherente con la marca.

User Experience se centra en el porqué y en el sentimiento del usuario. Es el proceso de crear productos que brindan experiencias significativas y relevantes. No se limita solo a lo que el usuario ve, sino a cómo se siente al interactuar con el sistema. Su objetivo principal es que el usuario encuentre una solución a su problema de la forma más fácil, lógica y satisfactoria posible.

Aunque ambas son disciplinas distintas, conviven juntas. Una interfaz hermosa (excelente UI) no sirve de nada si el sitio es difícil de usar y poco intuitivo (mal UX), y viceversa.

UI (Interfaz de Usuario)	UX (Experiencia de Usuario)
Se centra en el producto (el dispositivo).	Se centra en el usuario (la persona).
Es el medio: look & feel (aspecto y sensación).	Es el resultado: feeling (la reacción/sentimiento).
Trata sobre herramientas, botones, colores y tipografía.	Trata sobre usabilidad, análisis, lógica y satisfacción.
Es una parte de la experiencia de usuario.	Es el todo : el conjunto de factores de la interacción.
Su objetivo es la estética y la guía visual.	Su objetivo es la efectividad y la facilidad de uso.

PRODUCT MANAGER / PRODUCT OWNER

Product Manager (PM) → Es el "dueño del producto" ante el mercado. Su visión es a largo plazo y está orientada hacia el exterior (mercado y clientes).

- **Visión global:** Define la estrategia del producto y hacia dónde debe ir.
- **Negocio:** Se asegura de que el producto sea viable económicamente y cumpla los objetivos de la empresa.
- **Descubrimiento:** Habla con los clientes para entender sus problemas y decidir qué funcionalidades se deben construir.

Product Owner (PO) → Su visión es a corto/medio plazo y está orientada hacia el interior (el equipo de desarrollo). Es un rol específico dentro del marco de trabajo Scrum.

- **Ejecución:** Traduce la visión del PM en tareas concretas (Historias de Usuario).
- **Backlog:** Es el responsable de gestionar y priorizar la lista de tareas del equipo para que cada "Sprint" aporte valor.
- **Maximizar valor:** Se asegura de que el trabajo que hace el equipo técnico sea exactamente lo que se necesita.

SCRUM

Es un marco de trabajo (framework) ágil y ligero diseñado para el desarrollo y mantenimiento de productos. Se basa en un desarrollo iterativo e incremental donde el producto se construye en ciclos cortos llamados sprints. Hay tres roles claves:

PRODUCT OWNER → Es el enlace con el cliente y quien da la cara si no está el Product Manager. Valida y certifica las peticiones del cliente. Es el que crea y gestiona el Product backlog.

SCRUM MASTER → Actúa como organizador, moderador y facilitador. No da órdenes, sino que asegura que se cumplan las reglas del Scrum y la da a

conocer su organización. Controla el proceso, lee el Product Backlog y ayuda a definir el Scrum Backlog.

DEVELOPER TEAM → Ejecuta el desarrollo de los Sprints basándose en las directrices del Sprint Backlog. Está formado por analistas, diseñadores, programadores y testers, que trabajan juntos para lograr un producto funcional.

Los documentos más esenciales son:

PRODUCT BACKLOG → Lista completa y fija de funcionalidades y requisitos del cliente, sirve como un guión inicial. Es el inventario de todo lo que el producto podría necesitar.

SCRUM BACKLOG → Subconjunto de tareas extraídas del Product Backlog que el equipo se compromete a hacer en un ciclo concreto.

Las fases de un sprint son:

SPRINT PLANNING → El equipo y el Scrum Master planifican soluciones para la fase actual, mientras los analistas recogen requisitos y características del proyecto basándose en el Scrum Backlog.

DAILY MEETING → Reunión muy breve en la que el equipo planifica las tareas diarias y se analiza el proceso conseguido diariamente.

SPRINT REVIEW → Se realiza al finalizar el Sprint con el cliente y el equipo. Se presenta una reseña explicando qué tareas fueron realizadas y cuales no, para validar el cumplimiento de las tareas y planificar nuevas prioridades.

SPRINT RETROSPECTIVE → Reunión interna del equipo para dar feedback sobre el Sprint que acaba de terminar. Sirve para la mejora continua, analizando errores y aciertos para mejorar el proceso en el siguiente ciclo. Al finalizar, el equipo debe dejar por escrito compromisos y definiciones concretas de mejoras para el próximo sprint.

TIPOS DE PRUEBAS

Pruebas regresivas: Su objetivo es la estabilidad. Se realizan tras hacer un cambio o actualización para asegurar que no se ha estropeado nada de lo que ya funcionaba antes.

Pruebas de caja blanca: Se centran en el código fuente (el "cómo se hace"). Requieren conocer la programación para hacer debugging, revisando variables, sintaxis y lógica.

Pruebas de caja negra: Se centran en la aplicación (el "qué hace"), ignorando el código. Se prueban casos de uso y datos incorrectos desde la interfaz. Si la caja negra detecta un fallo, se recurre a la caja blanca para arreglarlo.

TIPOS DE CAJA BLANCA

- **De cubrimiento:** Su objetivo es que cada línea de código del programa se ejecute al menos una vez.
- **De condiciones:** Se aseguran de probar todos los resultados posibles (verdadero y falso) de cada sentencia condicional.
- **De bucles:** Comprueban los límites de las repeticiones probando el valor máximo, el máximo +1 y el máximo -1 para evitar bucles infinitos o errores de rango.

TIPOS DE CAJA NEGRA

- **De clases de equivalencia de datos:** Clasifican los datos en válidos e inválidos (ej. comprobar que una contraseña cumple o no los requisitos de longitud).
- **De valores límites:** Introducen datos irreales o extremos en formularios, como poner una edad de 150 años frente a una de 25, para ver cómo reacciona el sistema.
- **De interfaces:** Evalúan la GUI (interfaz gráfica) y su usabilidad, verificando atajos de teclado, ventanas emergentes y la navegación general.